Université de Caen Normandie

UFR des Sciences

Licence 3 Informatique

Projet de Conception Logicielle

Analyse des stratégies de coalitions dans le jeu de Tron

Équipe de développement :

N°	Prénom et nom	Numéro
1	Lamia **HADJ BENABDELMOULA**	22409436
2	Selssabil **KHELALFA**	22403233
3	Oumar **BAH**	22409261
4	Thierno Hammady **DIALLO**	22409301

Encadrant :

M. Alexis LECHERVY

Année universitaire 2025 – 2026

Table des matières

1 Introduction **3**
1.1 Contexte et motivation . . . 3
1.2 jusqu'à l'en-tête de donnéesion du projet . . . 3
1.3 Objectifs du projet . . . 3
1.4 Organisation du rapport . . . 3

2 Analyse des besoins **4**
2.1 Description générale du système . . . 4
2.2 Problématique . . . 4
2.3 Acteurs du système . . . 4
2.4 Exigences fonctionnelles . . . 4
2.5 Exigences non fonctionnelles . . . 5

3 Architecture du projet **5**
3.1 Architecture globale . . . 5
3.2 Organisation des packages . . . 5
3.3 Architecture MVC . . . 5
3.4 Interactions entre les modules . . . 7
3.4.1 Flux en mode interactif (GUI) . . . 7
3.4.2 Flux en mode expérimental . . . 7

4 Modélisation du système **7**
4.1 Pattern Observer . . . 7
4.2 Package modele . . . 7
4.2.1 Classe Plateau . . . 8
4.2.2 Classe Cellule . . . 8
4.2.3 Classe Position . . . 8
4.2.4 Classe ModeleJeu . . . 8

5 Package Stratégie IA et ses composantes **8**
5.1 Stratégie Minimax . . . 8
5.2 Stratégie Alpha-Beta . . . 9
5.3 Stratégie Paranoid . . . 10
5.4 Stratégie SOS . . . 11
5.5 Élagage dans MAXN . . . 12

6 Heuristiques **13**
6.1 Principe des heuristiques . . . 13
6.2 FreeSpace Heuristic . . . 13
6.3 Voronoi Heuristic . . . 14
6.4 Tree Of Chambers Heuristic . . . 15

7 Package view **16**
7.1 Architecture de l'interface d'affichage des expérimentations . . . 16
7.2 Architecture de l'interface des expérimentations . . . 17
7.2.1 Fonctionnalités . . . 19
7.2.2 Évolution . . . 19

8 Exploitation et Expérimentations **19**
8.1 Méthodes de lancement et Automatisation . . . 19
8.1.1 Script Shell d'Expérimentation Massive . . . 20
8.2 Interface console . . . 20
8.3 Interface graphique (GUI) . . . 21
8.4 Campagnes d'analyse et expérimentations . . . 21

9 Analyse des expérimentations **22**
9.1 Présentation des expériences . . . 22
9.2 Résultats globaux . . . 22
9.3 Analyse par stratégie . . . 22
9.3.1 MinMaxStrategie . . . 23
9.3.2 AlphaBetaStrategie . . . 23
9.3.3 MaxNStrategie . . . 23
9.3.4 ParanoidStrategie . . . 23
9.4 Impact des heuristiques . . . 24
9.5 Impact de la profondeur . . . 24
9.6 Impact de la taille du plateau . . . 25
9.7 Synthèse et réponse à la question scientifique . . . 26
9.8 Limites et perspectives . . . 26

10 Tests **27**
10.1 Stratégie de test . . . 27
10.2 Tests unitaires . . . 27
10.3 Validation . . . 27

11 Conclusion **27**
11.1 Bilan et Objectifs atteints . . . 27
11.2 Difficultés et Enseignements . . . 28
11.3 Perspectives . . . 28

1 Introduction

1.1 Contexte et motivation

Le jeu de Tron est un bon sujet pour l'intelligence artificielle, car il demande de prendre des décisions dans un environnement simple à représenter, mais qui change à chaque tour. Chaque joueur se déplace sur une grille de taille dynamique et laisse derrière lui une trace qui devient un obstacle permanent. Au fil de la partie, l'espace libre diminue, donc chaque déplacement devient important. Un simple mouvement peut protéger un joueur, enfermer un adversaire ou au contraire provoquer une défaite quelques tours plus tard. Sachant que les règles sont très simples, le joueur se déplace dans les quatre (4) directions et le but c'est d'être le dernier à survivre. Une collision avec un mur, avec l'adversaire, avec les obstacles créés dans la grille ou le bord du plateau entraînera une défaite.

Dans ce contexte, il ne suffit pas de survivre ; il faut aussi prévoir comment l'espace va évoluer et comment les autres joueurs vont réagir. Le projet devient encore plus intéressant quand plusieurs équipes ou plusieurs joueurs sont présents sur le même plateau, car on ne se limite plus à un simple duel. Selon la configuration choisie, certains comportements poussent à l'affrontement direct, tandis que d'autres ressemblent davantage à des alliances temporaires ou à une lutte pour contrôler l'espace. Le jeu permet ainsi d'observer concrètement comment des algorithmes de recherche se comportent lorsqu'ils doivent décider dans un contexte multi-agent.

1.2 jusqu'à l'en-tête de donnéesion du projet

Le projet réalisé est une application Java qui permet de simuler le jeu de Tron. L'application repose sur une architecture modulaire organisée autour d'un modèle de jeu, d'un contrôleur, d'une interface graphique Swing et d'un ensemble de stratégies d'intelligence artificielle. Elle permet de configurer une partie, de lancer des affrontements automatiques entre joueurs contrôlés par l'ordinateur, de visualiser leur déroulement, puis d'exploiter les résultats dans un cadre expérimental.

Le coeur du projet repose sur plusieursjusqu'à l'en-tête de donnéesjusqu'à l'en-tête de données stratégies de décision, comme **MinMax, AlphaBeta, MaxN, Paranoid, SOS** et une stratégie aléatoire utilisée pour comparer les résultats. Ces stratégies s'appuient sur différentes heuristiques, en particulier **FreeSpace, Voronoi** et **TreeOfChambers**. En complément de la partie interactive, le projet contient un module d'expérimentation capable d'exécuter un grand nombre de parties automatiques, d'exporter les résultats au format CSV et PDF, puis de générer des analyses et des graphiques.

1.3 Objectifs du projet

L'objectif principal était de créer un simulateur de Tron assez bien organisé pour servir à la fois de jeu et d'outil d'expérimentation. Il ne s'agissait donc pas seulement de faire un jeu qui fonctionne, mais aussi de proposer un cadre où plusieurs stratégies peuvent être comparées dans de bonnes conditions.

Avec ce projet, nous avons voulu étudier l'effet de la profondeur de recherche, du choix de l'heuristique, de la stratégie utilisée et de la taille du plateau sur le comportement des joueurs. Une attention particulière a été portée aux stratégies pensées pour des contextes multi-joueurs, en particulier MaxN, Paranoid et SOS, qui permettent d'élargir l'analyse au-delà du schéma classique à deux joueurs.

1.4 Organisation du rapport

Le rapport suit la structure du projet et de la démarche adoptée pendant sa réalisation. Il présente d'abord les besoins du projet, puis l'architecture choisie et les principaux choix de conception, les téchnologies et outils utilisés. Il détaille ensuite le modèle du jeu, les stratégies d'intelligence artificielle mises en place, les heuristiques utilisées et l'interface graphique. Une partie spécifique est consacrée aux expérimentations automatiques, à la méthodologie suivie, aux tests réalisés et à l'analyse des résultats obtenus. L'ensemble se termine par un bilan du travail accompli, les difficultés rencontrées et quelques pistes d'amélioration réalistes au regard de l'état actuel du projet.

2 Analyse des besoins

2.1 Description générale du système

Le système développé est un simulateur du jeu de Tron qui permet de faire jouer automatiquement ou visuellement plusieurs joueurs sur un plateau de taille dynamique. L'utilisateur configure une partie à partir d'une boîte de dialogue graphique. Il choisit le nombre de lignes et de colonnes, le nombre d'équipes, le nombre de joueurs par équipe, puis associe à chaque joueur une stratégie, une heuristique et une profondeur de recherche. Une fois la partie lancée, les joueurs sont placés sur le plateau, puis se déplacent tour après tour jusqu'à élimination de tous les joueurs sauf un, ou jusqu'à une situation de blocage conduisant à un match nul.

Le projet ne se limite pas à l'affichage d'une partie. Il inclut également un ensemble de classes dédiées aux expérimentations, à l'export des résultats, à leur lecture et à leur représentation graphique. Cette double fonction, à la fois interactive et expérimentale, explique l'organisation du code en plusieurs modules complémentaires.

2.2 Problématique

La problématique du projet est d'étudier le comportement de plusieurs algorithmes de décision dans le jeu de Tron. Le but est de comparer différentes stratégies afin de voir lesquelles sont les plus efficaces selon plusieurs paramètres, comme la profondeur de recherche, la taille du plateau et l'heuristique utilisée.

Dans ce rapport, l'analyse repose sur les expériences réellement lancées avec le script du projet. Il s'agit de **duels automatiques entre deux équipes composées d'un seul joueur**. Les tests ont été réalisés sur des plateaux de taille **10**, **15** et **20**, avec des profondeurs **3**, **6** et **9**. Les stratégies comparées sont **MinMaxStrategie, AlphaBetaStrategie, MaxNStrategie, ParanoidStrategie et SOSStrategie**. Les heuristiques testées sont **FreeSpaceHeuristic, VoronoiHeuristic** et **TreeOfChambers**. Chaque configuration a été jouée **50 fois**.

L'objectif est donc de voir si certaines stratégies deviennent meilleures quand la profondeur augmente, si le choix de l'heuristique change fortement les résultats, et si la taille du plateau influence les performances. Même si le projet permet aussi de créer des parties avec plus d'équipes et plus de joueurs, la partie expérimentale détaillée ici reste centrée sur les duels, car ce sont eux qui ont été réellement produits et analysés.

2.3 Acteurs du système

L'acteur principal du système est l'utilisateur qui configure la partie et observe son déroulement à travers l'interface graphique. Dans un contexte de projet universitaire, cet utilisateur peut être vu comme un expérimentateur qui cherche à comparer plusieurs stratégies dans des conditions identiques. Le système lui-même joue un second rôle important, puisqu'il automatise les tours de jeu, applique les règles, met à jour l'état du plateau, notifie l'interface des changements et enregistre les résultats lorsque les expériences sont lancées en série.

2.4 Exigences fonctionnelles

Sur le plan fonctionnel, l'application devait d'abord permettre de créer une partie configurable. La configuration réellement accessible par l'interface autorise une grille comprise entre 10 et 50 lignes et entre 10 et 50 colonnes. Le nombre d'équipes varie de 2 à 6, et le nombre de joueurs par équipe de 1 à 3. Chaque joueur peut recevoir une stratégie parmi Random, MinMax, AlphaBeta, MaxN, Paranoid et SOS, ainsi qu'une heuristique parmi FreeSpace, Voronoi et TreeOfChambers. La profondeur de recherche associée à chaque joueur peut être réglée entre 1 et 10.

Une fois la partie lancée, le système doit placer les joueurs, faire avancer les tours, détecter les collisions, mettre à jour l'état des joueurs et déterminer la fin de la partie. Le projet devait aussi proposer un mode d'expérimentation capable de lancer beaucoup de parties automatiquement, de regrouper les résultats, puis de les exporter. Enfin, l'interface devait permettre de visualiser la partie, de mettre en pause l'exécution, de contrôler la vitesse et de consulter un historique des affrontements réalisés.

2.5 Exigences non fonctionnelles

Au-delà des fonctionnalités visibles, plusieurs qualités logicielles étaient attendues. Le projet devait rester suffisamment modulaire pour permettre l'ajout de nouvelles stratégies ou de nouvelles heuristiques sans remise en cause complète de l'architecture. Il devait être assez robuste pour gérer différents scénarios de jeu et éviter les incohérences entre l'état logique du plateau et son affichage. La lisibilité du code et sa maintenabilité constituaient également des enjeux importants, d'où le choix d'une séparation par packages et l'utilisation d'une organisation inspirée du modèle MVC. La question des performances a aussi eu une place importante, surtout pour le mode expérimentation. Comme un grand nombre de parties doivent être jouées automatiquement, le projet utilise du multithreading afin de lancer plusieurs simulations en parallèle et de réduire le temps total d'exécution. Enfin, le recours à Java répondait à un besoin de portabilité et facilitait l'intégration de bibliothèques pour les tests, la génération de PDF et la visualisation de résultats.

3 Architecture du projet

3.1 Architecture globale

Le projet est organisé autour de plusieurs packages spécialisés qui correspondent aux grandes responsabilités de l'application. Le package `model` regroupe les classes représentant le plateau, les joueurs, les positions, les équipes, les directions, les états de cellule, les stratégies et les heuristiques. Le package `view` contient l'interface graphique Swing et les composants qui affichent le plateau, les contrôles et l'historique. Le package `controller` sert de lien entre les interactions utilisateur et le modèle. Le package `observer` fournit les interfaces et classes nécessaires à la notification de l'interface lorsqu'un changement intervient dans le modèle. Enfin, le package `experiment` rassemble les classes destinées à l'exécution en série des parties, à l'export des résultats et à leur analyse.

Cette organisation donne au projet une structure claire. Chaque zone du code répond à un rôle défini, ce qui rend l'ensemble plus lisible et facilite l'évolution du logiciel. Les classes principales restent concentrées dans le modèle, tandis que les outils annexes, comme l'analyse des résultats ou la production de graphiques, sont isolés dans un package dédié.

3.2 Organisation des packages

L'organisation des packages reflète directement les besoins du projet. Le package `controller` contient essentiellement les classes de pilotage et d'historique. Le package `experiment` est plus dense, car il regroupe à la fois la configuration d'une campagne, l'exécution des parties, la représentation des résultats, les exporteurs CSV et PDF, ainsi que les classes chargées d'analyser les fichiers générés. Le package `model` est le plus central, puisqu'il contient les entités de base du jeu et tout ce qui relève de la prise de décision algorithmique. Le package `test` reprend cette organisation en proposant des tests ciblés sur plusieurs stratégies et heuristiques. Enfin, le package `view` rassemble les panneaux graphiques qui composent la fenêtre principale.

Cette répartition correspond aux fichiers réellement présents dans le projet. Elle montre que le logiciel a été pensé pour distinguer clairement le jeu lui-même, son interface et le versant expérimental.

3.3 Architecture MVC

L'application suit une logique proche du patron MVC. Le modèle correspond aux classes qui décrivent et font évoluer l'état du jeu, en particulier `ModeleJeu`, `Plateau`, `Player` et les classes de stratégie. La vue est constituée par les composants Swing, en particulier `GUI`, `GameBoardPanel`, `SidePanel`, `ControlSection` et `SpeedSection`. Le contrôleur principal, `GameController`, coordonne la création d'une partie, la relance d'une simulation, la lecture de l'historique et le déclenchement des actions utilisateur.

Ce découpage n'est pas purement théorique. Il se retrouve dans la manière dont le programme a été construit. Les règles du jeu et les calculs de stratégie ne dépendent pas directement des composants graphiques, ce qui permet de réutiliser le modèle dans les expérimentations sans passer systématiquement par l'interface. Cette séparation constitue un point fort du projet.

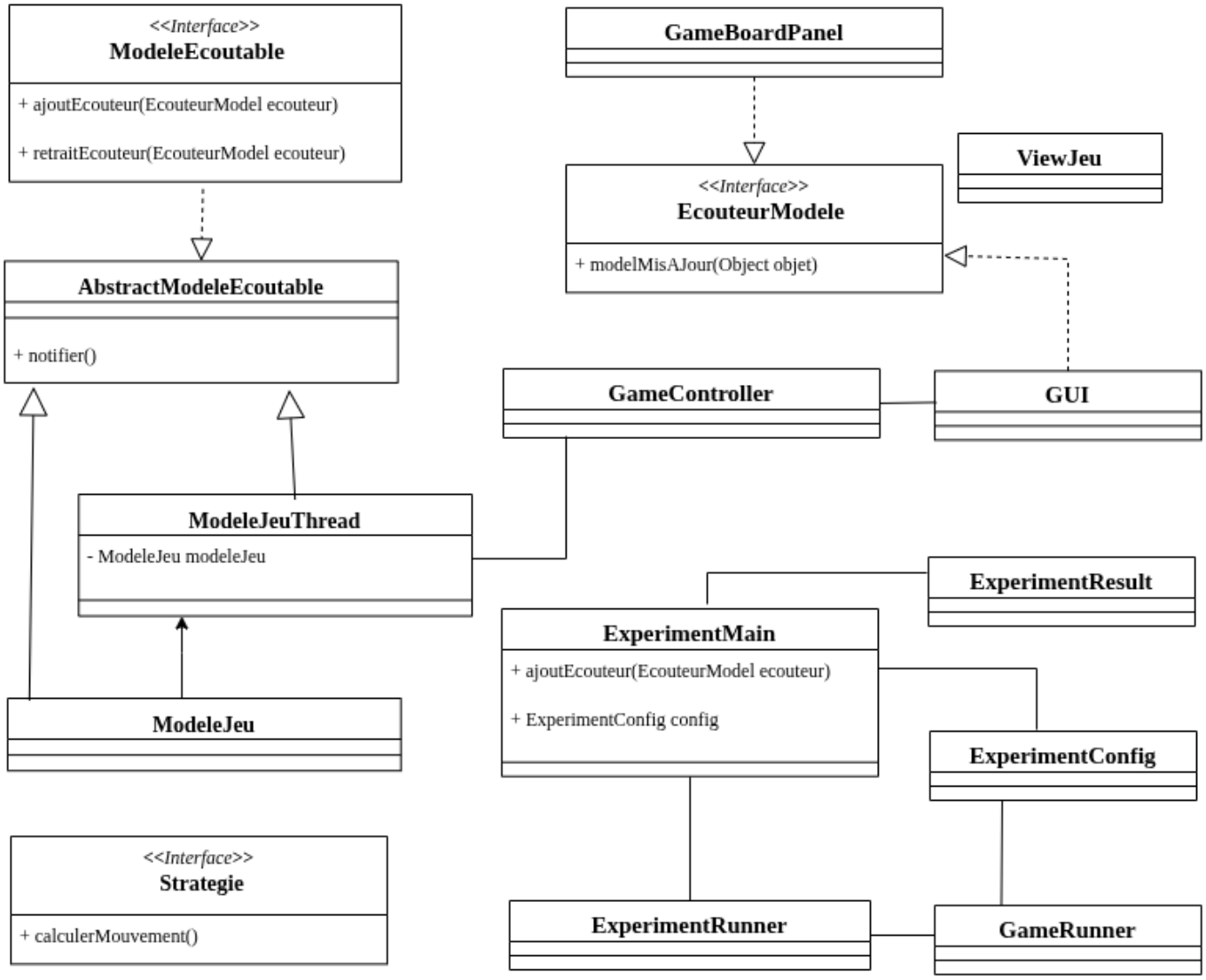


FIGURE 1 – Le diagramme UML des classes principales

3.4 Interactions entre les modules

L'un des points forts de cette architecture est l'indépendance totale du **Modèle**. Celui-ci contient l'intégralité des règles et de la logique de simulation, lui permettant de fonctionner de manière autonome, sans nécessiter de contrôleur ou d'interface graphique.

3.4.1 Flux en mode interactif (GUI)

Lorsqu'une partie est lancée via l'interface utilisateur, le processus d'interaction suit une chaîne précise :

1. **Configuration :** Le contrôleur récupère les paramètres (taille du plateau, types d'IA) depuis la boîte de configuration.
2. **Initialisation :** Il instancie les objets `Player`, crée le `ModeleJeu` et lie ce dernier aux panneaux graphiques (`GameBoardPanel`, etc.).
3. **Cycle de jeu :** À chaque tour, le modèle calcule ou reçoit les mouvements, met à jour l'état du `Plateau`, puis notifie automatiquement la vue via le pattern Observer pour déclencher le rafraîchissement visuel.

3.4.2 Flux en mode expérimental

Dans le cadre du package `experiment`, l'enchaînement est analogue mais optimisé pour la performance brute. Le pilotage est assuré par les classes `ExperimentMain`, `ExperimentRunner` et `GameRunner`.

— **Exécution en série :** Ces classes instancient les joueurs et exécutent les parties en boucle sans aucune intervention de la `View`.

— **Collecte de données :** Les résultats de chaque simulation sont encapsulés dans des objets `GameResult` et agrégés dans un `ExperimentResult`.

— **Exploitation :** En fin de chaîne, les modules d'export et d'analyse transforment ces données brutes en fichiers CSV, rapports PDF ou graphiques statistiques.

Cette séparation permet d'utiliser le même moteur de règles pour deux usages radicalement différents : une démonstration visuelle fluide et une analyse statistique de masse.

4 Modélisation du système

4.1 Pattern Observer

Le patron Observer est utilisé pour relier le modèle à la vue. Les classes du package `observer`, telles que `ModeleEcoutable`, `AbstractModeleEcoutable` et `EcouteurModele`, permettent au modèle de notifier les composants intéressés lorsqu'une modification survient. Ainsi, lorsqu'un tour de jeu est exécuté et que le plateau change, la vue peut être rafraîchie automatiquement sans devoir interroger en permanence l'état du modèle.

Ce choix simplifie la synchronisation entre la logique de jeu et l'affichage. Il évite également de coupler trop fortement le modèle aux composants Swing, ce qui reste conforme à l'esprit d'une architecture modulaire.

4.2 Package modele

Le modèle du jeu repose sur une représentation discrète de l'espace. Le plateau est une grille composée de cellules identifiées par des positions. Chaque joueur occupe une position courante et appartient à une équipe. À chaque tour, il choisit une direction parmi les déplacements autorisés. Le déplacement a deux conséquences : la position du joueur change, et la case quittée devient une trace assimilée à un obstacle. Ce mécanisme suffit à créer la dynamique caractéristique de Tron, où l'espace se ferme progressivement.

Les décisions de déplacement dépendent de la stratégie associée au joueur. La stratégie peut se contenter d'un choix aléatoire, ou bien lancer une recherche de type MinMax, AlphaBeta, MaxN, Paranoid ou SOS. Dans tous les cas, l'évaluation intermédiaire des positions repose sur une heuristique. Le modèle combine donc deux niveaux : la stratégie décide comment explorer les coups possibles, tandis que l'heuristique donne une valeur aux positions intermédiaires.

4.2.1 Classe Plateau

La classe `Plateau` représente la surface de jeu. Elle stocke les dimensions de la grille, la structure des cellules et l'information nécessaire pour savoir si une case est libre, occupée ou interdite. Elle joue un rôle central, car toutes les stratégies utilisent le plateau pour connaître les coups possibles, simuler des mouvements ou évaluer l'espace encore libre. Dans le déroulement d'une partie, le plateau constitue donc la mémoire de l'environnement.

Son importance apparaît aussi dans les heuristiques. FreeSpace évalue les zones atteignables à partir du plateau courant, Voronoi s'appuie sur les distances aux cases libres pour répartir le territoire, et TreeOf-Chambers exploite la topologie de l'espace lorsqu'il se fragmente en régions. Le plateau ne sert donc pas seulement à l'affichage ; il soutient directement l'intelligence décisionnelle du système.

4.2.2 Classe Cellule

La classe `Cellule` permet de modéliser l'état élémentaire d'une case du plateau. Elle sert à distinguer les cases libres des cases déjà occupées ou transformées en obstacles. Associée à l'énumération `CellState`, elle fournit une représentation simple mais suffisante pour gérer l'évolution du jeu. Cette granularité est adaptée à Tron, où la notion de collision se réduit essentiellement à la question suivante : la case ciblée est-elle encore disponible au moment du déplacement.

4.2.3 Classe Position

La classe `Position` encapsule les coordonnées d'une case et fournit des opérations utiles, comme le calcul de distance de Manhattan ou la comparaison entre positions. Cette abstraction évite de manipuler partout de simples couples d'entiers. Elle améliore la lisibilité du code et facilite le traitement algorithmique, en particulier lorsque les stratégies génèrent des coups voisins, comparent des distances ou vérifient qu'une position appartient à la grille.

4.2.4 Classe ModeleJeu

`ModeleJeu` constitue la classe centrale du projet. Elle orchestre l'évolution d'une partie en maintenant le plateau, la liste des joueurs, l'état de vie de chacun, le déroulement des tours et les conditions d'arrêt. C'est elle qui applique réellement les déplacements et qui détermine si une collision entraîne l'élimination d'un joueur. Son rôle est également de servir de point d'appui à la vue et au contrôleur.

La classe concentre la logique métier du jeu. Elle ne se contente pas de stocker des données ; elle exprime la manière dont la partie progresse. C'est pourquoi la plupart des autres composants, qu'il s'agisse des stratégies, de l'interface ou des expérimentations, se structurent autour d'elle.

5 Package Stratégie IA et ses composantes

Le projet distingue clairement la stratégie de l'heuristique. Une stratégie définit la façon dont un joueur explore les coups possibles avant de prendre une décision. Une heuristique, quant à elle, sert à évaluer la qualité d'un état lorsque la profondeur maximale de recherche est atteinte ou lorsqu'il n'est pas pertinent d'explorer davantage. Cette séparation est incarnée dans le code par les interfaces et classes abstraites du package `model`, en particulier `Strategie`, `AbstractStrategie` et `Heuristic`.

Toutes les stratégies mises en place répondent au même objectif immédiat, à savoir choisir une direction de déplacement valable. En revanche, elles diffèrent par leur manière de représenter les adversaires et de raisonner sur l'avenir. Certaines suivent une logique classique de maximisation et de minimisation, alors que d'autres sont pensées pour le multi-joueur ou pour des comportements de coalition.

5.1 Stratégie Minimax

La stratégie Minimax cherche le coup qui donne la meilleure valeur au joueur courant en supposant qu'au niveau suivant le jeu évolue de la manière la plus défavorable. Dans le projet, la classe `MinMaxStrategie` copie

le plateau, simule chaque déplacement possible, puis lance une recherche récursive jusqu'à une profondeur donnée. Quand la profondeur est atteinte, ou quand il n'y a plus de coup possible, l'état est évalué avec l'heuristique choisie.

Le fonctionnement peut être résumé par le pseudo-code suivant.

Algorithm 1: Algorithme Minimax

```
minimax(plateau, player, depth, maximisant)
if Temps écoulé ou depth == 0 ou getCoupsPossibles(pos) est vide then
    return heuristic.evaluate(plateau, player)
if maximisant then
    bestValue ← −∞
    for chaque direction dir dans getCoupsPossibles(pos) do
        copiePlateau ← copierPlateau(plateau)
        deplacer(copiePlayer, dir, copiePlateau)
        value ← minimax(copiePlateau, copiePlayer, depth - 1, faux)
        bestValue ← max(bestValue, value)
    return bestValue
else
    worstValue ← +∞
    for chaque direction dir dans getCoupsPossibles(pos) do
        copiePlateau ← copierPlateau(plateau)
        deplacer(copiePlayer, dir, copiePlateau)
        value ← minimax(copiePlateau, copiePlayer, depth - 1, vrai)
        worstValue ← min(worstValue, value)
    return worstValue
```

Cette stratégie reste surtout adaptée aux situations proches du duel, car elle alterne simplement entre une phase où l'on maximise et une phase où l'on minimise.

5.2 Stratégie Alpha-Beta

La stratégie Alpha-Beta reprend l'idée de Minimax, mais elle coupe certaines branches de recherche dès qu'il devient clair qu'elles ne pourront pas améliorer le résultat final. Dans le code, la classe `AlphaBetaStrategie` commence par simuler chaque déplacement du joueur, cherche ensuite un adversaire vivant sur le plateau, puis lance une recherche récursive avec deux bornes, `alpha` et `beta`. Cette version est utilisée surtout dans le cas d'un affrontement simple.

Le pseudo-code correspondant est le suivant.

Algorithm 2: Minimax avec élagage Alpha-Beta

```
minimaxAlphaBeta(plateau, me, opponent, depth, α, β, maximizing)
if temps écoulé OR depth == 0 OR terminal node then
    return heuristic.evaluate(plateau, me)
if maximizing then
    value ← −∞
    for chaque direction dir in coups do
        simulation ← copier(plateau)
        simulerMouvement(simulation, me, dir)
        eval ← minimaxAlphaBeta(simulation, me, opponent, depth - 1, α, β, false)
        value ← max(value, eval)
        α ← max(α, value)
        if β ≤ α then
            break                                          // Élagage Alpha
    return value
else
    value ← +∞
    for chaque direction dir in coups do
        simulation ← copier(plateau)
        simulerMouvement(simulation, opponent, dir)
        eval ← minimaxAlphaBeta(simulation, me, opponent, depth - 1, α, β, true)
        value ← min(value, eval)
        β ← min(β, value)
        if β ≤ α then
            break                                          // Élagage Beta
    return value
```

L'avantage principal de cette stratégie est de garder une logique proche de Minimax tout en évitant une partie des calculs inutiles.

5.3 Stratégie Paranoid

La stratégie Paranoid suppose que tous les autres joueurs cherchent ensemble à faire perdre le joueur courant. Dans le projet, la classe `ParanoidStrategie` récupère d'abord la liste des joueurs encore vivants. Elle teste ensuite chaque coup du joueur principal, puis alterne entre une étape où ce joueur cherche à maximiser sa valeur et une étape où l'ensemble des autres joueurs cherche à la minimiser. Le principe reste proche d'Alpha-Beta, mais la partie minimisante parcourt tous les adversaires.

Le pseudo-code peut être écrit ainsi.

Algorithm 3: Algorithme Paranoid avec élagage Alpha-Beta

```
paranoid(plateau, me, depth, α, β, maximiserJoueur)
if depth == 0 OR NOT me.isAlive() then
    return heuristic.evaluate(plateau, me)
if maximiserJoueur then
    maxVal ← −∞
    coups ← plateau.getCoupsPossibles(me.pos)
    if coups est vide then
        return heuristic.evaluate(plateau, me)
    for chaque direction dir dans coups do
        plateauCopie ← simulerCoup(plateau, me, dir)
        val ← paranoid(plateauCopie, me, depth - 1, α, β, false)
        maxVal ← max(maxVal, val)
        α ← max(α, maxVal)
        if β ≤ α then
            break
    return maxVal
else
    minVal ← +∞
    foreach joueur p dans joueurs do
        if p == me OR NOT p.isAlive() then
            continue
        for chaque direction dir dans p.getCoupsPossibles() do
            plateauCopie ← simulerCoup(plateau, p, dir)
            val ← paranoid(plateauCopie, me, depth - 1, α, β, true)
            minVal ← min(minVal, val)
            β ← min(β, minVal)
            if β ≤ α then
                break                                   // Élagage paranoïaque
    return minVal
```

Cette manière de faire est simple à interpréter : le joueur agit comme s'il était seul contre tous.

5.4 Stratégie SOS

La stratégie SOS est pensée pour les situations multi-joueurs où l'on veut conserver une évaluation pour chaque joueur au lieu de ramener tout le monde à un simple duel. Dans le projet, la classe `SOSStrategie` teste les coups possibles du joueur de départ, puis lance une recherche récursive qui retourne un vecteur de scores. À chaque niveau, le joueur courant choisit la branche qui améliore le plus sa propre composante dans ce vecteur.

Le pseudo-code simplifié est le suivant.

C'est une variante intéressante ! Ton algorithme SOS (ou Socially Optimal Strategy / Simple Opponent Search) pour les jeux multi-joueurs ne retourne plus une valeur unique, mais un vecteur d'utilité (un score par joueur). Ici, le joueur courant cherche à maximiser sa propre composante dans ce vecteur.

Algorithm 4: Algorithme SOS (Socially Optimal Strategy)

```
sos(plateau, depth, player)
if depth == 0 OR coupsPossibles est vide then
    evaluation ← heuristic.evaluate(plateau, player)
    return utiliteSociale(evaluation)
id ← joueurs.indexOf(player)
bestScore ← −∞
bestVecteur ← utiliteSociale(−∞)
for chaque direction dir dans plateau.getCoupsPossibles() do
    plateauCopie ← copier(plateau)
    simulerMouvement(plateauCopie, player, dir)
    currentVecteur ← sos(plateauCopie, depth - 1, nextPlayer(playerCopie))
    if currentVecteur[id] > bestScore then
        bestScore ← currentVecteur[id]
        bestVecteur ← currentVecteur
return bestVecteur
```

Dans cette version, la fonction d'utilité sociale recopie la même valeur pour tous les joueurs. L'intérêt de cette stratégie est surtout de poser une base pour raisonner autrement qu'avec un simple max contre min.

5.5 Élagage dans MAXN

La stratégie MaxN est utilisée quand plusieurs joueurs prennent part au jeu et que chacun cherche à améliorer sa propre valeur. Dans le projet, la classe `MaxNStrategie` utilise un parallélisme à la racine : chaque coup possible du joueur de départ est évalué dans une tâche séparée, puis l'algorithme récursif `maxN` retourne un vecteur contenant une valeur pour chaque joueur. Le joueur courant choisit toujours la branche qui maximise sa propre case dans ce vecteur.

Le pseudo-code simplifié de cette stratégie est le suivant.

Algorithm 5: Algorithme MaxN pour n joueurs

```
maxN(plateau, players, depth, currentIndex)
if depth == 0 OR timeExceeded() then
    return evaluateAll(plateau, players)
current ← players.get(currentIndex)
if NOT current.isAlive() then
    return maxN(plateau, players, depth, (currentIndex + 1) % players.size())
coups ← plateau.getCoupsPossibles(current.pos)
shuffle(coups)
if coups est vide then
    return evaluateAll(plateau, players)
bestVector ← null
bestValue ← −∞
for chaque direction dir dans coups do
    grid ← copier(plateau)
    simulerMouvement(grid, current, dir)
    nextIndex ← (currentIndex + 1) % players.size()
    values ← maxN(grid, nextPlayers, depth - 1, nextIndex)
    if values[currentIndex] > bestValue then
        bestValue ← values[currentIndex]
        bestVector ← values
return bestVector
```

Dans cette stratégie, il n'y a pas un élagage Alpha-Beta classique. En revanche, le projet essaie quand même de réduire le coût en limitant le parallélisme au premier niveau de l'arbre.

6 Heuristiques

6.1 Principe des heuristiques

Dans un jeu comme Tron, il est impossible d'explorer tous les coups possibles dès que le plateau ou le nombre de joueurs devient plus grand. Les heuristiques jouent donc un rôle central. Elles permettent d'estimer la qualité d'une position intermédiaire en attribuant une note à l'état du plateau. Cette note sert ensuite de base aux stratégies pour comparer les coups possibles lorsqu'elles atteignent leur profondeur limite.

Le projet implémente trois heuristiques distinctes. Leur coexistence permet d'étudier non seulement la qualité intrinsèque d'une stratégie de recherche, mais aussi la façon dont elle dépend d'une bonne évaluation locale des états.

6.2 FreeSpace Heuristic

L'heuristique FreeSpace mesure simplement combien de cases libres restent accessibles à partir de la position du joueur. Dans le projet, cette estimation est obtenue avec un parcours en largeur à partir de la case courante. Si le joueur est déjà éliminé, une très mauvaise valeur est renvoyée immédiatement. Cette heuristique est simple, rapide et facile à comprendre.

Le pseudo-code correspondant est le suivant.

Algorithm 6: Heuristique d'Espace Libre (BFS)

```
evaluate(plateau, joueur) if NOT joueur.isAlive() then
    return −1000000
return countAccessibleCells(plateau, joueur.pos)
countAccessibleCells(plateau, positionInitiale) if NOT plateau.estDansPlateau(positionInitiale) then
    return 0
compteur ← 0
visite ← Matrice booléenne [L][C] initialisée à false
file ← File vide
file.enfiler(positionInitiale)
visite[positionInitiale.lig][positionInitiale.col] ← true
while file n'est pas vide do
    courant ← file.defiler()
    compteur ← compteur + 1
    foreach direction dir dans {HAUT, BAS, GAUCHE, DROITE , NONE} do
        voisin ← courant.deplacer(dir)
        if plateau.estDansPlateau(voisin) AND NOT visite[voisin.lig][voisin.col] then
            if plateau.estLibre(voisin) then
                visite[voisin.lig][voisin.col] ← true
                file.enfiler(voisin)
return compteur
```

Cette méthode donne une bonne première approximation, mais elle ne distingue pas vraiment un espace sécurisé d'un espace qui sera bientôt perdu.

6.3 Voronoi Heuristic

L'heuristique Voronoi cherche à estimer le territoire que chaque joueur pourra contrôler. Pour cela, elle lance une propagation simultanée à partir de la position de tous les joueurs vivants. Chaque case libre est attribuée à l'équipe du joueur qui l'atteint le plus vite. Si deux équipes arrivent exactement en même temps, la case est marquée comme contestée et n'est attribuée à personne. Le score final correspond à la différence entre le territoire de l'équipe du joueur et celui des autres équipes.

Le pseudo-code simplifié est le suivant.

Algorithm 7: Heuristique de Voronoi (Partition du Territoire)

```
evaluate(plateau, joueur) if NOT joueur.isAlive() then
    return −1000000
L ← plateau.nbLignes                                   // Initialisation des structures
C ← plateau.nbColonnes
distance ← Tableau de taille L × C rempli de ∞
proprietaireEquipe ← Tableau de taille L × C rempli de null
file ← File vide
// Identification des sources (têtes des joueurs)
foreach case p du plateau do
    occupant ← plateau.getJoueurAt(p)
    if occupant ≠ null then
        distance [p] ← 0
        proprietaireEquipe [p] ← occupant.team
        file.enfiler(p)
// Propagation multi-source (BFS)
while file n'est pas vide do
    courant ← file.defiler()
    equipeCourante ← proprietaireEquipe [courant]
    if equipeCourante == null then
        continue
    // Case contestée
    foreach direction dir do
        voisin ← courant.deplacer(dir)
        if plateau.estDansPlateau(voisin) AND plateau.estLibre(voisin) then
            if distance [voisin] == ∞ then
                distance [voisin] ← distance [courant] +1
                proprietaireEquipe [voisin] ← equipeCourante
                file.enfiler(voisin)
            else if distance [voisin] == distance [courant] +1 AND proprietaireEquipe [voisin] ≠
             equipeCourante then
                proprietaireEquipe [voisin] ← null                // Égalité : case contestée
// Calcul du score final
scoreEquipe ← 0, scoreAdversaires ← 0
foreach case c dans proprietaireEquipe do
    if c == joueur.team then
        scoreEquipe ← scoreEquipe + 1
    else if c ≠ null then
        scoreAdversaires ← scoreAdversaires + 1
return (scoreEquipe − scoreAdversaires)
```

Cette heuristique est plus riche que FreeSpace, car elle tient compte de la concurrence entre joueurs pour l'occupation de l'espace.

6.4 Tree Of Chambers Heuristic

L'heuristique Tree Of Chambers essaie de mieux comprendre la structure du plateau quand l'espace commence à se couper en zones séparées. Dans le projet, cette heuristique suit plusieurs étapes. Elle commence par repérer l'adversaire le plus proche, construit ensuite une partition de type Voronoi à deux joueurs, puis

cherche les points d'articulation, c'est-à-dire les cases qui relient plusieurs régions. À partir de là, elle estime la taille de la chambre principale et la meilleure chambre voisine accessible.

Le pseudo-code simplifié ci-dessous reprend cette logique générale.

Algorithm 8: Heuristique Tree of Chambers (ToC)

```
evaluate(plateau, joueur) if NOT joueur.isAlive() then
    return −10^6
adversaires ← plateau.getOpponents(joueur)
monScore ← 0, scoreAdverse ← 0
foreach adversaire opp do
    V ← calculerVoronoi2P(plateau, joueur, opp)
    monScore ← max(monScore, tocValue(V, joueur))
    scoreAdverse ← max(scoreAdverse, tocValue(V, opp))
return monScore − scoreAdverse
tocValue(voronoi, p) region ← voronoi.getRegion(p)
articul ← findArticulationPoints(region)
// Chambre principale (contenant la tête du joueur)
main ← exploreChamber (region, articul, p.pos)
bestAdj ← 0
foreach point a ∈ main.pointsArticulation do
    ch ← exploreChamber (region, articul, a)
    if NOT ch.touchesBattlefront then
        bestAdj ← max(bestAdj, ch.size)
return main.size + bestAdj
```

Cette heuristique est plus coûteuse, mais elle essaie de mieux représenter les situations où le plateau est découpé en couloirs et en zones presque fermées.

7 Package view

Le package `view` regroupe l'ensemble des composants graphiques du projet. L'interface a été développée en **Swing** autour d'une fenêtre principale et de plusieurs panneaux spécialisés. L'objectif n'était pas seulement de proposer un affichage du jeu, mais aussi de fournir un véritable espace de configuration et d'observation. La classe `GUI` centralise l'assemblage de ces composants. L'interface permet ainsi de lancer une partie, d'en suivre l'évolution visuellement et d'agir sur certains paramètres d'exécution comme la vitesse ou la pause.

Cette interface joue un rôle important dans la compréhension du projet, car elle rend immédiatement visibles les effets des stratégies mises en place. Elle transforme donc le simulateur en outil pédagogique, mais aussi en support d'expérimentation.

7.1 Architecture de l'interface d'affichage des expérimentations

L'interface graphique a été développée en utilisant **Swing** et **JFreeChart** afin d'offrir une visualisation claire et interactive des résultats des expérimentations. Elle est organisée autour d'une fenêtre principale et de plusieurs panneaux spécialisés. Cette structure permet de séparer les zones d'affichage, de configuration et de contrôle.

La classe `GameBoardPanel` est le composant chargé de représenter graphiquement le plateau et les déplacements des joueurs. Elle affiche les cellules, les traces laissées sur la grille ainsi que les positions courantes des agents. Son fonctionnement repose sur les notifications envoyées par le modèle, ce qui permet de mettre à jour l'affichage au fur et à mesure que la partie progresse. Ce panneau rend la dynamique du jeu plus lisible et permet de comprendre rapidement si une stratégie adopte un comportement d'évitement, d'occupation du territoire ou de confrontation directe.

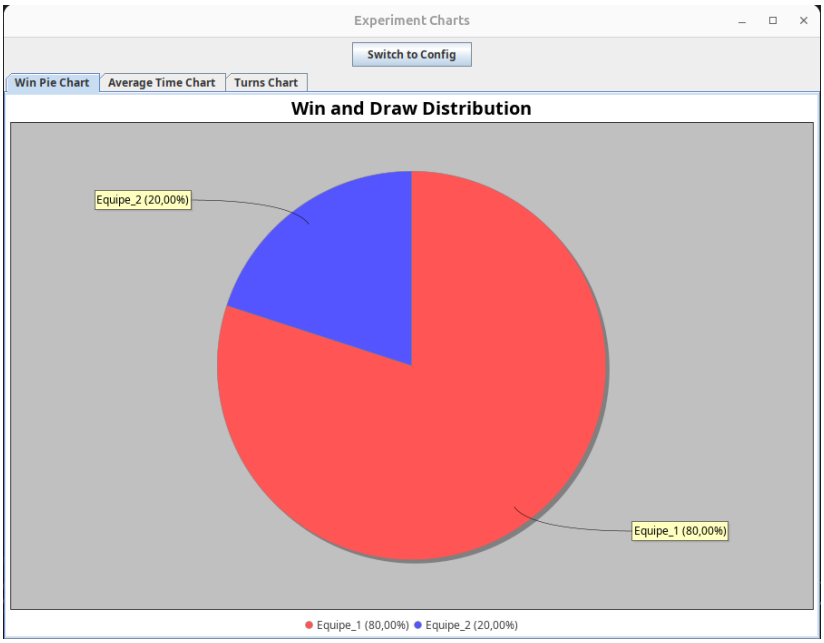
La classe `SidePanel` regroupe les éléments annexes à l'affichage principal. Elle sert de conteneur aux différentes sections de contrôle et d'information. Cette organisation améliore l'ergonomie générale de la fenêtre en séparant clairement la zone de jeu de la zone de commande. Ce choix de structuration se retrouve dans l'ensemble du package `view` et permet d'ajouter ou de modifier une partie de l'interface sans devoir réécrire entièrement la fenêtre principale.

La classe `ControlSection` rassemble les commandes de pilotage de la partie, en particulier le lancement, la pause et la relance. Ce composant joue un rôle pratique essentiel, car il relie directement l'utilisateur au contrôleur. Il participe à la dimension interactive du projet et facilite les tests manuels des stratégies.

La classe `SpeedSection` permet d'ajuster la vitesse d'exécution de la simulation. Cette fonctionnalité est particulièrement utile pour observer le comportement des algorithmes. Une vitesse élevée convient aux tests rapides, tandis qu'une exécution plus lente rend possible une analyse visuelle plus fine des déplacements et des décisions successives.

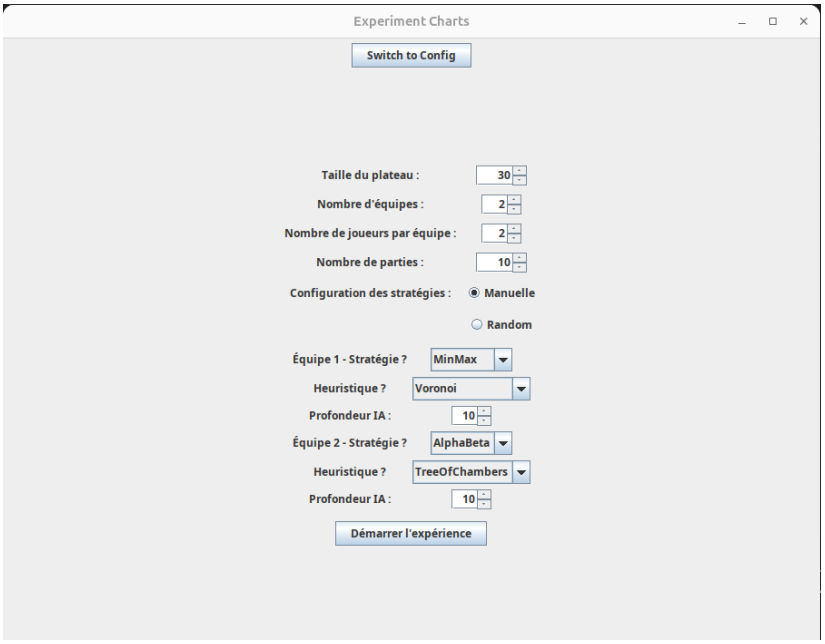
7.2 Architecture de l'interface des expérimentations

En complément de l'interface du jeu, une interface dédiée aux expérimentations a également été développée. Elle avait pour objectif de faciliter le lancement des campagnes de tests et l'observation des résultats obtenus. La fenêtre principale sert de conteneur pour tous les panneaux et composants de cette interface. Elle offre une vue d'ensemble des résultats des expérimentations et permet de naviguer entre les différentes sections de configuration et de visualisation.



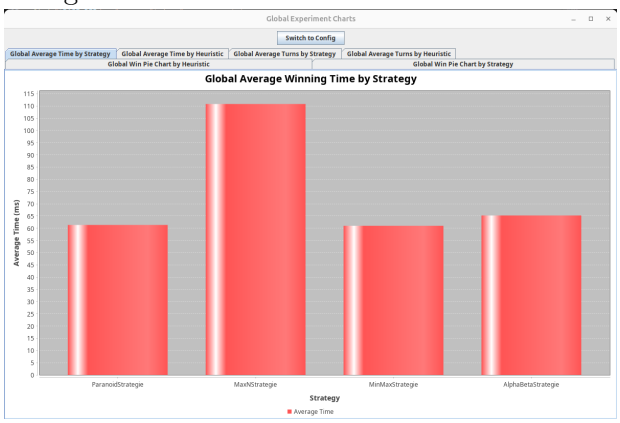
Exemple de fenêtre principale de l'interface

Une partie importante de cette interface est consacrée à la configuration. Elle permet de définir les paramètres des expérimentations, comme la taille du plateau, le nombre d'équipes, les stratégies, les heuristiques et le nombre de parties à exécuter. Cette partie donne donc une certaine souplesse pour préparer plusieurs campagnes de tests sans devoir modifier directement le code.

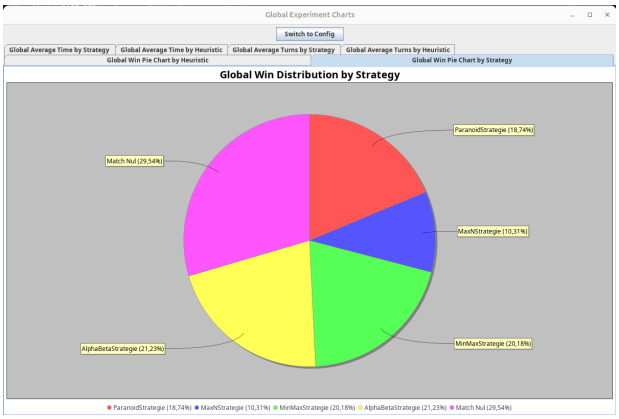


Exemple de panneau de configuration

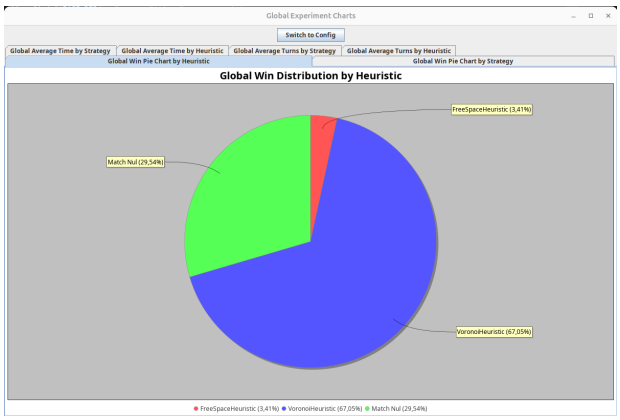
L'interface comporte également une partie dédiée à la visualisation. Celle-ci s'appuie sur **JFreeChart** pour afficher les résultats sous forme de graphiques, par exemple des diagrammes circulaires ou des histogrammes. Ce choix rend la lecture des statistiques plus simple et permet de comparer plus rapidement les configurations testées.



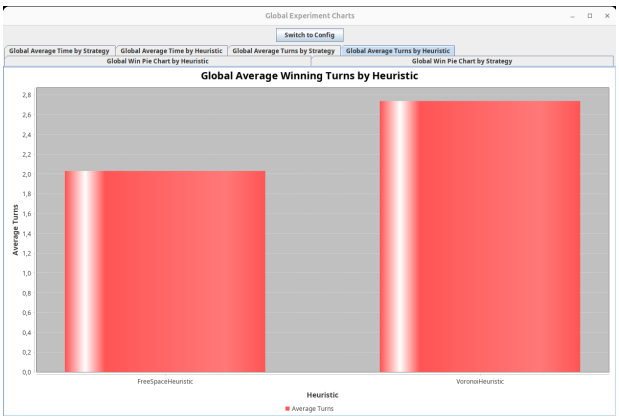
(a) Exemple de graphique statistique



(b) Visualisation des résultats par configuration



(c) Exemple de comparaison graphique



(d) Autre exemple d'affichage des résultats

FIGURE 2 – Exemples d'écrans de l'interface d'analyse des expérimentations

7.2.1 Fonctionnalités

L'outil d'analyse permettait d'abord de lancer les expérimentations automatiquement à partir des paramètres choisis. Une fois la configuration définie, les parties étaient exécutées en arrière-plan, ce qui évitait de devoir les lancer une par une manuellement. Cela rendait possible l'exécution d'un grand nombre de tests dans un temps raisonnable.

Il permettait ensuite d'afficher les résultats obtenus de manière lisible. Les données produites après les expériences étaient regroupées sous forme de statistiques et de graphiques. Cela aidait à comparer plus facilement les stratégies, les heuristiques, les profondeurs de recherche ou encore les tailles de plateau.

Enfin, l'outil facilitait aussi la gestion des configurations. Il était possible de modifier les paramètres des expériences, de relancer de nouvelles campagnes de tests et d'essayer différentes combinaisons de stratégies et d'heuristiques. Cette souplesse permettait d'explorer plusieurs scénarios sans avoir à changer manuellement tout le code à chaque fois.

7.2.2 Évolution

Cette interface d'affichage a constitué notre première approche pour la visualisation des données et des statistiques. Cependant, nous avons rencontré certaines limites en termes de performance et de capacité de traitement, notamment à cause de la complexité des données et du temps de calcul nécessaire pour les expérimentations à grande échelle.

Ces difficultés nous ont conduits à explorer une solution complémentaire en utilisant un code **Python** pour le traitement des données et la génération de graphiques. Cette approche s'est révélée plus adaptée pour analyser un grand volume de résultats. Nous avons néanmoins conservé cette partie dans le projet : même si elle ne correspond pas à l'outil principal retenu pour l'analyse finale, elle reste fonctionnelle et fait partie du travail réalisé.

8 Exploitation et Expérimentations

Cette section détaille les différentes manières d'interagir avec le logiciel, depuis l'utilisation interactive jusqu'à l'analyse automatisée de données de masse.

8.1 Méthodes de lancement et Automatisation

Le projet s'appuie sur l'outil d'automatisation **Apache Ant** pour gérer le cycle de vie de l'application. Un fichier de configuration `build.xml` définit plusieurs cibles (*targets*) permettant de compiler, tester et lancer le projet de manière standardisée.

Le projet dispose d'un système d'automatisation à deux niveaux : **Ant** pour la gestion du cycle de vie logiciel (compilation, JAR, tests) et un **Script Shell** pour le pilotage intensif des campagnes d'expérimentation.

L'outil Ant permet non seulement de compiler, mais aussi de personnaliser les lancements via des arguments système. Les cibles principales sont :

- `ant compile` : Prépare l'arborescence de construction (`build/`) et compile l'intégralité des sources Java ainsi que les classes de test en respectant.
- `ant test` : Exécute de manière automatisée la suite de tests unitaires JUnit. Les résultats sont exportés dans un dossier de rapport dédié.
- `ant dist` : Génère une archive exécutable `.jar` dans le dossier `dist/`. Cette cible inclut également la génération de la **Javadoc** pour documenter le projet.
- `ant run` : La cible par défaut. Elle compile le projet, génère le JAR et lance l'exécution principale.
- `ant clean` : Nettoie le répertoire de travail en supprimant les fichiers compilés, les distributions et la documentation générée.

8.1.1 Script Shell d'Expérimentation Massive

Pour évaluer l'efficacité des algorithmes, un script Shell (`script.sh`) a été développé. Il permet d'automatiser des milliers de duels en faisant varier systématiquement tous les paramètres du moteur de jeu.

— **Exploration combinatoire :** Le script itère sur plusieurs tailles de plateaux (10×10 à 20×20), plusieurs profondeurs de recherche (3, 6, 9) et croise toutes les stratégies (*Minimax, AlphaBeta, MaxN, Paranoid et SOS*) avec toutes les heuristiques (*Voronoi, Tree of Chambers, etc.*).

— **Gestion des données :** Pour chaque configuration, 50 parties sont jouées. Le script agrège les résultats dans un fichier **CSV** et fusionne les rapports PDF individuels.

Avantage notable : Cette automatisation permet de transformer des données brutes en une étude comparative rigoureuse sans intervention humaine, facilitant ainsi l'identification des configurations d'IA les plus performantes.

8.2 Interface console

L'interface en ligne de commande sert de base technique au projet. Elle permet de suivre le déroulement d'une partie via des logs textuels détaillés. En mode console, ce qui facilite le débuggage rapide de la logique de mouvement et des scores sans la charge processeur liée au rendu graphique.

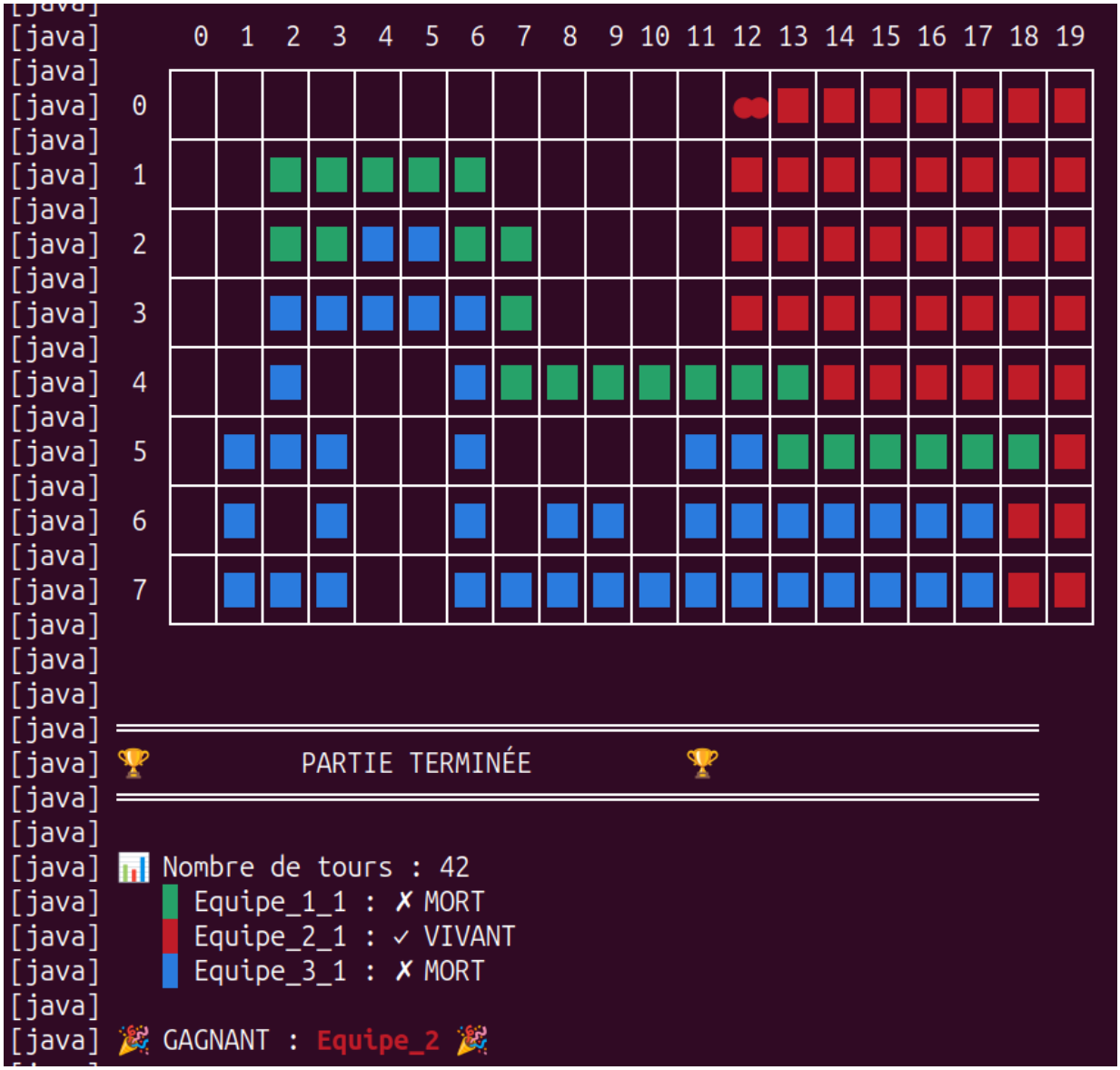


FIGURE 3 – Une illustration d'une partie en mode console

8.3 Interface graphique (GUI)

Développée avec la bibliothèque **Swing**, l'interface graphique offre une immersion complète dans le jeu. Elle s'organise autour de plusieurs zones complémentaires. Au centre, le plateau de jeu est représenté sous la forme d'une grille dynamique qui affiche les joueurs, leurs positions respectives ainsi que les murs laissés au fil des déplacements. Cette représentation permet de suivre visuellement l'évolution de la partie et de mieux comprendre le comportement des différentes stratégies.

Autour de cette zone principale, l'interface intègre également un panneau de contrôle qui permet de démarrer, mettre en pause ou réinitialiser une partie. Ce panneau donne aussi accès à certains réglages, comme la vitesse de simulation, ce qui rend les tests plus souples et plus faciles à observer. Enfin, l'interface affiche des informations complémentaires en temps réel, notamment l'historique récent de la partie, certains résultats intermédiaires et différents éléments utiles au suivi des actions effectuées pendant le jeu.

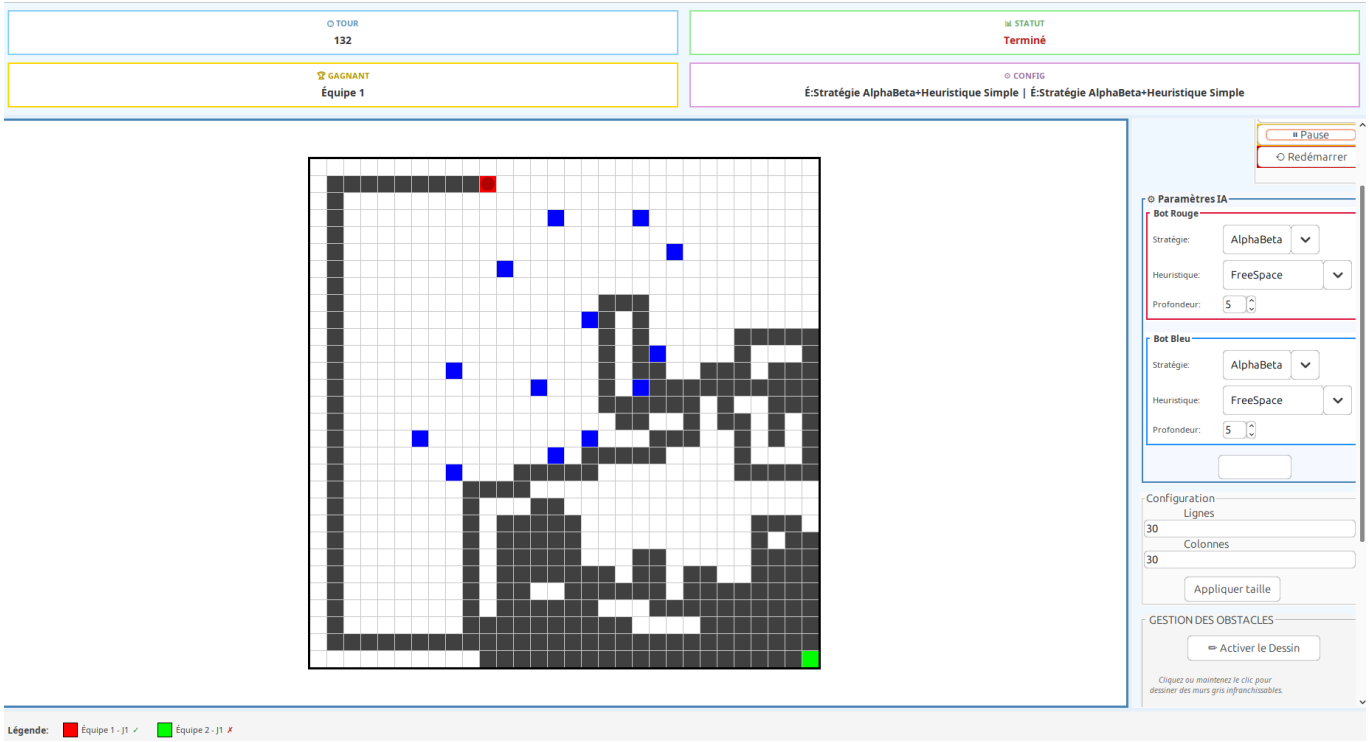


FIGURE 4 – Aperçu de l'interface graphique Swing

8.4 Campagnes d'analyse et expérimentations

Le volet expérimental constitue une partie importante du projet, car il permet d'évaluer les performances des différentes stratégies dans un cadre automatisé. Il repose principalement sur le package `experiment`, qui prend en charge l'organisation des campagnes de tests, l'exécution des parties et la récupération des résultats.

Le protocole expérimental consiste à lancer un grand nombre de parties entre différentes configurations d'algorithmes. Cela permet de comparer plusieurs stratégies, comme `MinMaxStrategie`, `MaxNStrategie`, `ParanoidStrategie` ou `SOSStrategie`, ainsi que plusieurs heuristiques d'évaluation. Chaque partie produit ensuite un objet `GameResult`, qui contient les informations essentielles pour l'analyse, comme l'équipe gagnante, le nombre de tours joués et le temps moyen de calcul.

Une fois les résultats récupérés, ils sont exportés dans des fichiers CSV, puis exploités pour construire des graphiques et des tableaux comparatifs. Cette étape rend possible une analyse plus globale des performances observées, notamment pour étudier l'effet de la profondeur de recherche, du choix de l'heuristique ou encore de la taille du plateau sur le comportement des intelligences artificielles.

9 Analyse des expérimentations

9.1 Présentation des expériences

L'analyse présentée ici repose sur les résultats obtenus après les campagnes d'expérimentations automatiques lancées dans le projet. Les données ont ensuite été regroupées et visualisées sous forme de graphiques afin de mieux comparer les stratégies, les heuristiques, la profondeur de recherche et la taille du plateau.

Les graphiques permettent de voir rapidement les tendances générales. Ils ne servent pas seulement à compter le nombre de victoires, mais aussi à observer l'effet de certains paramètres sur le comportement global des stratégies. Dans cette partie, nous nous appuyons donc sur les résultats effectivement produits par le projet et sur les graphiques générés à partir des fichiers de sortie.

```
Temps      : 1h 46m 30s
Configs    : 1620 / 2025
Parties    : 81000
CSV        : /home/lamia/Documents/13mars/projet2-jeuxdetorne-khelalfa-hadj-bena
bdelmoula-diallo-bah/csv/duel_20260326_193013.csv
PDF        : /home/lamia/Documents/13mars/projet2-jeuxdetorne-khelalfa-hadj-bena
bdelmoula-diallo-bah/pdf/duel_20260326_193013.pdf
═══════════════════════════════════════════

Aperçu des résultats (premières lignes):
# EXPÉRIENCES DUEL - jeu. 26 mars 2026 19:30:13 CET
# Plateau: 10 15 20, Profondeurs: 3 6 9
# Stratégies: MinMaxStrategie AlphaBetaStrategie MaxNStrategie ParanoidStrategie
 SOSStrategie
# Heuristiques: FreeSpaceHeuristic VoronoiHeuristic TreeOfChambers
# Parties par config: 50
```

9.2 Résultats globaux

Dans l'ensemble, les résultats montrent que les performances ne dépendent pas d'un seul paramètre. Le choix de la stratégie a un effet important, mais il ne peut pas être séparé du choix de l'heuristique. On observe aussi que certaines profondeurs donnent de meilleurs résultats que d'autres, alors que la taille du plateau semble avoir un impact plus limité sur les tendances générales.

Les graphiques montrent surtout que les écarts les plus visibles viennent de la stratégie utilisée et de la qualité de l'heuristique. Certaines combinaisons donnent des résultats nettement meilleurs, alors que d'autres restent plus irrégulières ou produisent davantage de matchs nuls.

9.3 Analyse par stratégie

L'analyse par stratégie permet de comparer plus précisément le comportement de chaque algorithme. Le graphique suivant résume les résultats obtenus selon la stratégie utilisée.

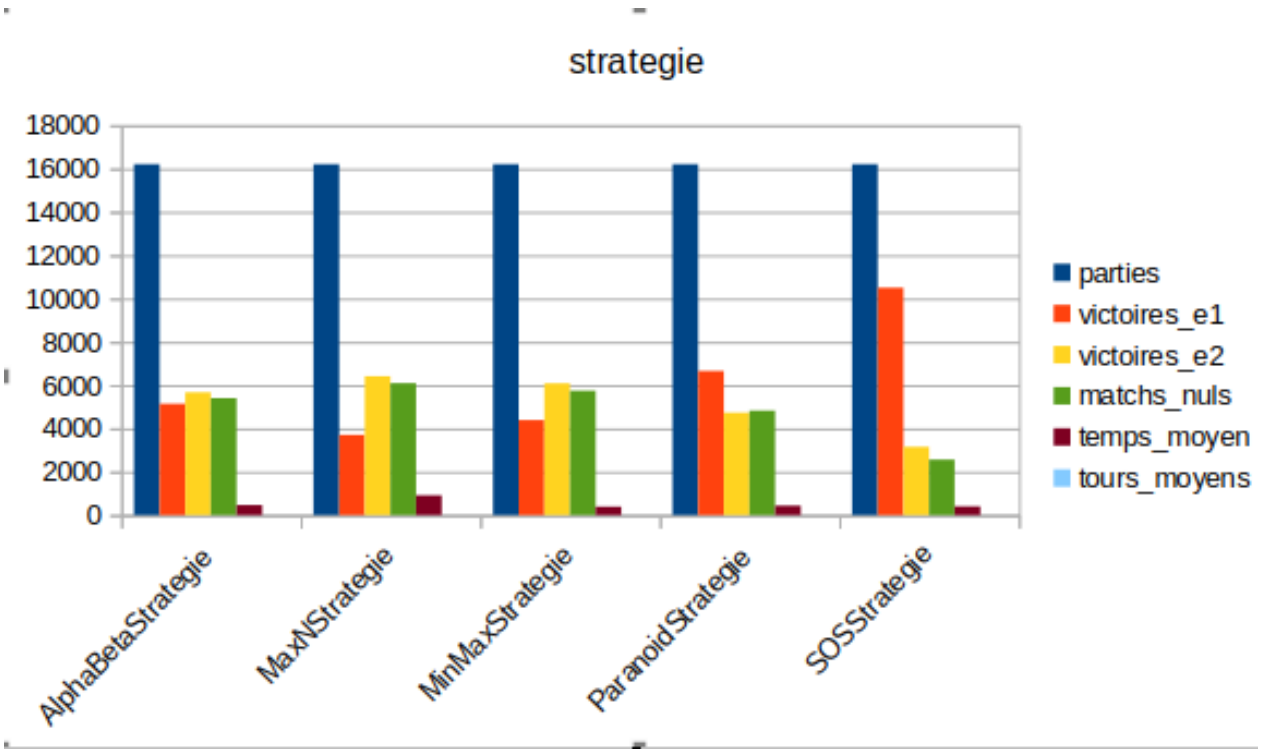


FIGURE 5 – Comparaison globale des stratégies

Sur la figure 5, on remarque que les stratégies ne se comportent pas toutes de la même manière. Certaines gagnent plus souvent, d'autres produisent plus de matchs nuls, et certaines semblent plus dépendre du contexte. Le nombre de parties jouées est comparable entre les stratégies, ce qui permet de faire une comparaison assez équilibrée.

9.3.1 MinMaxStrategie

MinMaxStrategie donne des résultats globalement corrects, sans être la stratégie la plus dominante. Elle obtient un nombre de victoires raisonnable, mais reste derrière les meilleures configurations observées. Son comportement paraît assez stable, avec peu de résultats extrêmes. Cela confirme que MinMax reste une base classique et sérieuse, mais que son efficacité dépend beaucoup de l'évaluation utilisée.

9.3.2 AlphaBetaStrategie

AlphaBetaStrategie donne elle aussi des résultats solides. Son intérêt principal est de conserver une logique proche de MinMax tout en réduisant le coût de calcul grâce à l'élagage. Dans les résultats observés, elle reste compétitive et garde un bon niveau général. Elle apparaît comme une stratégie équilibrée, capable de produire de bons résultats sans être systématiquement la meilleure dans tous les cas.

9.3.3 MaxNStrategie

MaxNStrategie obtient des résultats plus mitigés. Le graphique montre qu'elle peut gagner dans certains cas, mais qu'elle reste globalement moins convaincante que les stratégies les plus performantes. Ce résultat n'est pas forcément surprenant, car MaxN est surtout pensé pour des contextes multi-joueurs. Dans un cadre plus restreint ou plus compétitif, son comportement semble moins directement efficace.

9.3.4 ParanoidStrategie

ParanoidStrategie ressort comme l'une des stratégies les plus fortes dans les résultats observés. Elle obtient un nombre de victoires élevé et semble mieux transformer les parties en résultats favorables. Son approche plus prudente et plus défensive paraît bien fonctionner dans le jeu de Tron, où la gestion de l'espace est essentielle. Cette stratégie semble donc particulièrement intéressante dans les configurations testées.

9.4 Impact des heuristiques

L'effet des heuristiques est très visible dans les résultats. Le graphique ci-dessous montre clairement que toutes les heuristiques n'apportent pas le même niveau de qualité aux stratégies.

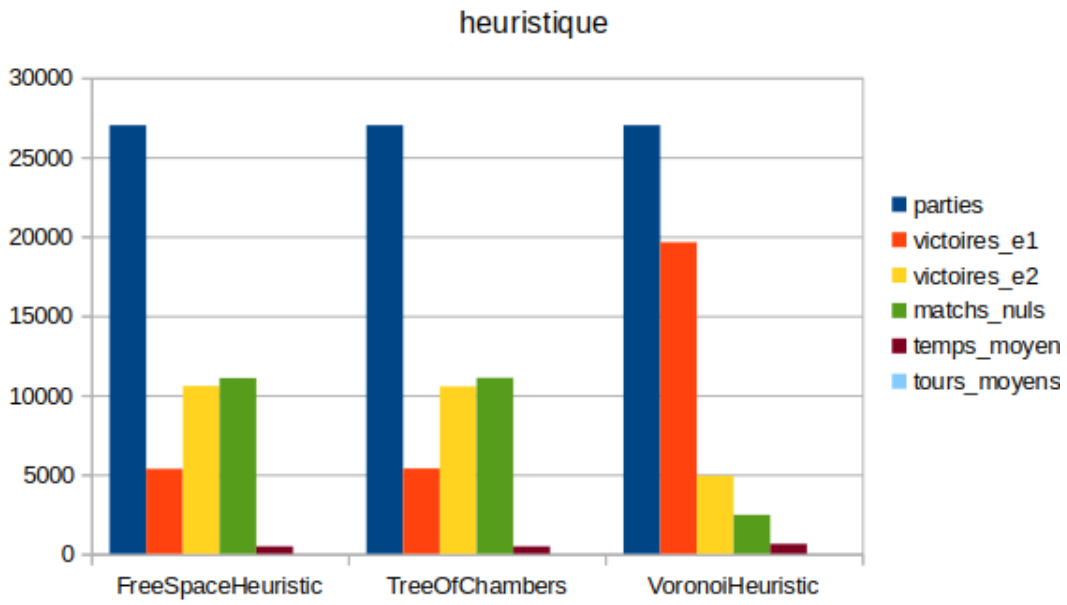


FIGURE 6 – Comparaison des heuristiques selon les stratégies

La figure 6 met en évidence une tendance très nette : **VoronoiHeuristic** donne de meilleurs résultats que les autres heuristiques dans presque tous les cas. Elle se détache clairement pour chaque stratégie affichée. À l'inverse, **FreeSpaceHeuristic** et **TreeOfChambers** donnent des résultats plus faibles et souvent assez proches l'une de l'autre.

Cela montre que l'heuristique joue un rôle central. Même une bonne stratégie perd une grande partie de son intérêt si la fonction d'évaluation utilisée n'est pas assez pertinente. Dans ce projet, Voronoi semble être l'heuristique la plus adaptée pour représenter le contrôle de l'espace sur le plateau.

9.5 Impact de la profondeur

La profondeur de recherche a également une influence sur les résultats. Le graphique suivant permet de comparer les performances observées selon les différentes profondeurs testées.

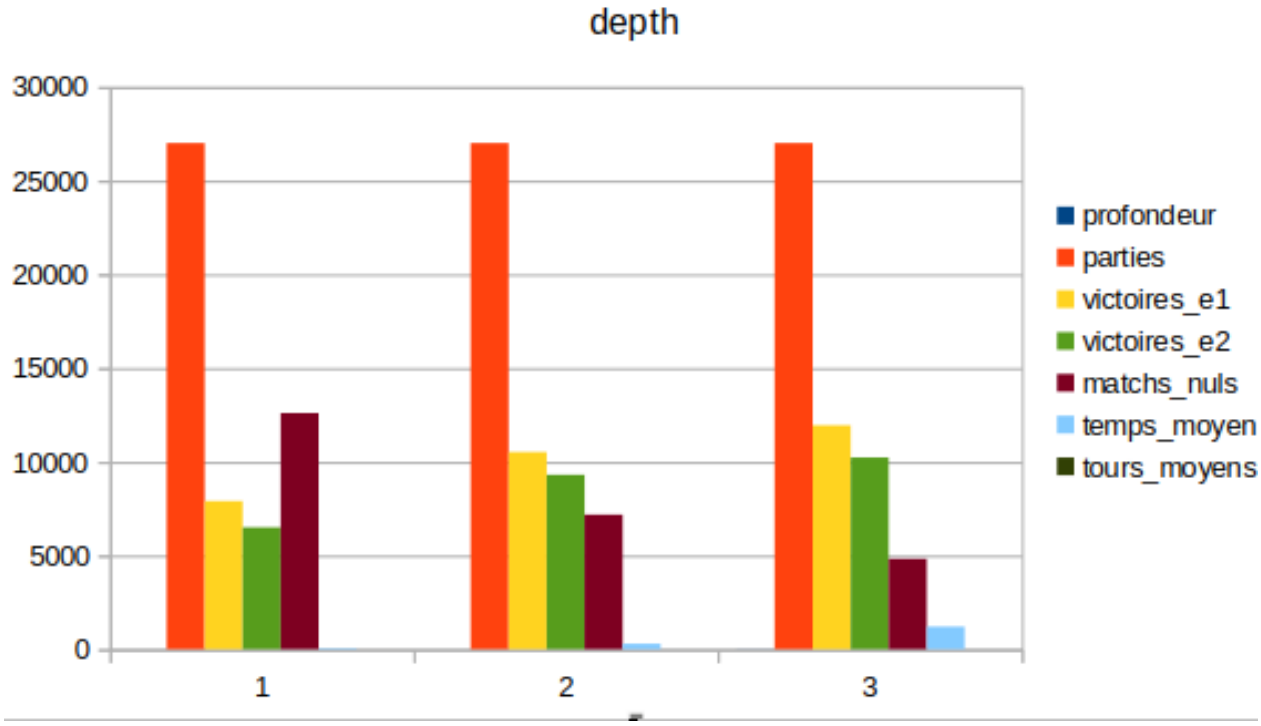


FIGURE 7 – Impact de la profondeur de recherche

La figure 7 montre que la profondeur a bien un effet, mais que cet effet n'est pas totalement linéaire. La profondeur la plus élevée testée dans ce graphique semble produire davantage de victoires et moins de matchs nuls que les autres. En revanche, elle s'accompagne aussi d'un coût plus important en temps moyen et en nombre de tours moyens.

Cela veut dire qu'augmenter la profondeur peut améliorer la qualité de décision, mais que cela augmente aussi le coût de calcul. Le gain n'est donc pas gratuit. Dans le cadre du projet, on peut dire qu'une profondeur plus grande peut être utile, mais seulement si le temps d'exécution reste acceptable.

9.6 Impact de la taille du plateau

La taille du plateau modifie elle aussi le comportement global des parties, mais l'effet visible dans les résultats semble moins marqué que celui de la stratégie ou de l'heuristique.

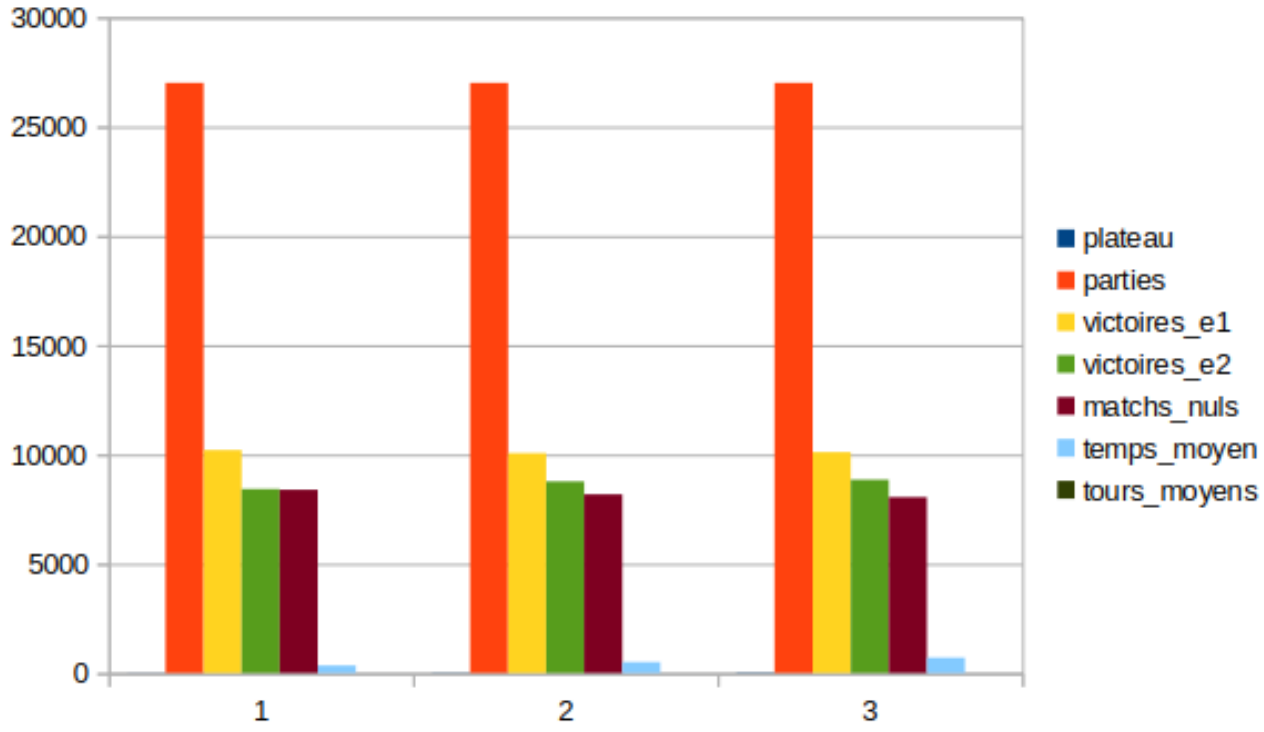


FIGURE 8 – Impact de la taille du plateau

Sur la figure 8, les résultats restent assez proches d'un plateau à l'autre. Le nombre de parties, de victoires et de matchs nuls ne change pas fortement entre les trois cas affichés. En revanche, on remarque que le temps moyen et le nombre moyen de tours ont tendance à augmenter lorsque la taille du plateau augmente. Cela est logique, car un plateau plus grand laisse plus d'espace aux joueurs et peut prolonger les affrontements.

On peut donc dire que la taille du plateau joue surtout sur la durée et le rythme de la partie, mais qu'elle ne change pas complètement la hiérarchie générale observée entre les stratégies et les heuristiques.

9.7 Synthèse et réponse à la question scientifique

Les résultats permettent de dégager plusieurs conclusions simples. D'abord, le facteur le plus visible dans cette campagne d'expérimentation est l'heuristique. Le graphique des heuristiques montre clairement que **VoronoiHeuristic** domine les autres. Ensuite, du côté des stratégies, **ParanoidStrategie** ressort comme l'une des plus efficaces dans les configurations observées.

L'analyse de la profondeur montre aussi qu'une recherche plus poussée peut améliorer les résultats, mais au prix d'un temps de calcul plus élevé. Enfin, la taille du plateau semble avoir un effet plus limité sur les performances globales, même si elle influence la durée des parties.

La réponse à la question scientifique peut donc être formulée ainsi : dans ce projet, la profondeur de recherche a bien une influence, mais son effet reste moins important que celui de la stratégie et surtout de l'heuristique. Une profondeur plus grande peut aider, mais elle n'est réellement intéressante que si elle s'appuie sur une bonne évaluation du plateau.

9.8 Limites et perspectives

Les résultats présentés dans cette partie doivent avant tout être compris comme des résultats **indicatifs**. Ils permettent de dégager des tendances générales et de comparer plusieurs comportements, mais ils ne suffisent pas encore à établir des conclusions définitives sur toutes les dimensions du projet. En effet, les expériences réalisées ici ne couvrent qu'une partie des possibilités offertes par le jeu et par les stratégies implémentées.

Pour aller plus loin, il serait nécessaire d'enrichir le protocole expérimental. Par exemple, il serait intéressant de tester plus précisément l'impact de la **taille des équipes** et du **nombre total de joueurs** sur les performances observées. Cela permettrait de mieux comprendre comment les stratégies évoluent lorsque la

dynamique de jeu devient réellement multi-joueurs. Il serait également pertinent d'étudier plus directement les **alliances** et les comportements de coalition, en particulier pour les stratégies comme SOS, qui prennent tout leur sens dans ce type de contexte.

D'autres améliorations seraient aussi possibles, comme l'ajout de nouvelles profondeurs de recherche, l'étude de davantage de tailles de plateau, ou encore la mise en place d'outils d'analyse plus précis pour comparer les résultats. Ainsi, cette première campagne d'expérimentations constitue surtout une base de travail solide, mais elle demande encore à être approfondie pour exploiter pleinement tout le potentiel du projet.

10 Tests

10.1 Stratégie de test

La stratégie de test adoptée dans le projet repose principalement sur des tests unitaires ciblant le modèle, les heuristiques et plusieurs stratégies d'intelligence artificielle. L'objectif était de vérifier le comportement général des composants avant de les intégrer dans des parties complètes. Ce choix est cohérent avec la structure du projet, car la majorité de la logique critique se trouve dans le package `model`.

Les tests ne remplacent évidemment pas l'observation d'une partie réelle ni l'analyse expérimentale, mais ils constituent une première garantie de bon fonctionnement pour les briques de base.

10.2 Tests unitaires

Le package `test` contient en particulier des classes comme `ModeleJeuTest`, `MinMaxStrategieTest`, `AlphaBetaStrategieTest`, `ParanoidStrategieTest`, `SOSStrategieTest`, `FreeSpaceHeuristicTest`, `VoronoiHeuristicTest` et `TreeOfChambersHeuristicTest`. L'ensemble est exécuté via JUnit 4, présent dans le répertoire `lib`. Cette couverture montre que le projet ne s'est pas limité à une validation visuelle du jeu, mais a intégré un minimum d'automatisation dans la vérification du code.

Les tests portent en particulier sur la validité des directions proposées, la cohérence de certaines évaluations heuristiques et le comportement global du modèle dans des situations simples. Ils offrent une base utile, même si certaines propriétés plus complexes du jeu restent difficiles à vérifier uniquement par des tests unitaires.

10.3 Validation

La validation du projet s'est faite à deux niveaux. D'une part, les tests unitaires ont permis de contrôler plusieurs composants isolés. D'autre part, l'exécution de parties via l'interface et via le module d'expérimentation a rendu possible une validation empirique de l'ensemble. Cette double approche est importante dans un projet de simulation, car un algorithme peut être correct au niveau syntaxique tout en produisant un comportement peu crédible si l'intégration générale du système est défaillante.

11 Conclusion

11.1 Bilan et Objectifs atteints

Le projet a abouti à une plateforme de simulation complète, alliant rigueur logicielle et capacités expérimentales. Les objectifs initiaux ont été pleinement remplis :

- **Architecture robuste :** Une séparation stricte (MVC/Observer) permettant d'isoler la logique métier du rendu graphique.
- **Intelligence Artificielle :** Implémentation fonctionnelle de plusieurs algorithmes de recherche (*Minimax, Alpha-Beta, MaxN, Paranoid, SOS*) et d'heuristiques spatiales (*Voronoi, Tree of Chambers*).
- **Pipeline de données :** Capacité à générer des campagnes d'expérimentation massives, de l'exécution brute à l'analyse graphique automatisée.

11.2 Difficultés et Enseignements

Le développement a révélé des défis techniques et conceptuels majeurs :

— **Complexité algorithmique :** Maintenir une architecture propre tout en intégrant des logiques de recherche variées a nécessité une abstraction poussée des stratégies.

— **Rigueur expérimentale :** L'aspect aléatoire des positions de départ a imposé la mise en place d'un protocole de duel strict (50 parties par configuration) pour obtenir des statistiques significatives.

— **Optimisation :** La gestion du multithreading pour les simulations intensives a représenté un défi technique pour garantir l'isolation des données et la cohérence des résultats.

11.3 Perspectives

Le moteur de jeu constitue une base solide pour des évolutions futures :

— **Extension multi-agent :** Approfondir les tests en configuration "équipes contre équipes" pour évaluer la coopération entre IA.

— **Affinement du protocole :** Réduire l'influence de l'aléatoire en utilisant des scénarios de départ prédéfinis.

— **Heuristiques avancées :** Intégrer de nouveaux modèles d'évaluation basés sur l'apprentissage ou une analyse topologique encore plus fine du plateau.

Ce projet démontre qu'au-delà du jeu, le Tron est un excellent support pour l'étude des systèmes multi-agents et des stratégies de contrôle spatial en temps limité.

Références

[1] Russell, S., Norvig, P. (2010). *Artificial Intelligence : A Modern Approach*. Prentice Hall.

[2] Sturtevant, N. (2003). *Last-Branch and Speculative Pruning Algorithms for Maxn*. AAAI.

[3] Kang, L. (2022). *Heuristic Tree of Chambers for the Game of Tron*. Maastricht University. `https://project.dke.maastrichtuniversity.nl/games/files/bsc/Kang_Bsc-paper.pdf`

[4] Tron Game. *Wikipedia*. `https://en.wikipedia.org/wiki/Tron_(video_game)`

[5] Voronoi Diagram. *Wikipedia*. `https://en.wikipedia.org/wiki/Voronoi_diagram`